

Efficient Construction of Model Family through Progressive Training Using Model Expansion

Kazuki Yano[†], Sho Takase^{†,‡}, Sosuke Kobayashi[†], Shun Kiyono[‡], Jun Suzuki[†].

[†]Tohoku University

[‡]SB Intuitions

yano.kazuki@dc.tohoku.ac.jp

{sho.takase, shun.kiyono}@sbintuitions.co.jp

{sosuke.kobayashi.b2, jun.suzuki}@tohoku.ac.jp

Abstract

As Large Language Models (LLMs) gain widespread practical application, providing the model family of different parameter sizes has become standard practice to address diverse computational requirements. Conventionally, each model in a family is trained independently, resulting in computational costs that scale additively with the number of models. We propose an efficient method for constructing the model family through progressive training, where smaller models are incrementally expanded to larger sizes to create a complete model family. Through extensive experiments with a model family ranging from 1B to 8B parameters, we demonstrate that our method reduces computational costs by approximately 25% while maintaining comparable performance to independently trained models. Furthermore, by strategically adjusting maximum learning rates based on model size, our method outperforms the independent training across various metrics. Beyond performance gains, our approach offers an additional advantage: models in our family tend to yield more consistent behavior across different model sizes.

1 Introduction

As Large Language Models (LLMs) gain widespread practical application, providing models with (i) a consistent architecture and (ii) different parameter sizes (hereafter referred to as a **model family**) has become standard practice in the NLP community. For example, Llama 3.1 consists of models with 8B, 70B, and 405B parameters (AI@Meta, 2024), and Gemma 3 includes 1B, 4B, 12B and 27B parameters (GemmaTeam, 2025). Similarly, Qwen2.5 provides a model family with 0.5B, 1.5B, 3B, 7B, 14B, 32B and 72B parameters (Yang et al., 2024b).

Offering a model family is a typical approach to addressing diverse computational requirements and use cases. Smaller models enable efficient processing and rapid response for daily tasks while facilitating deployment in environments with strict computational requirements, such as smartphones and edge devices (Abdin et al., 2024). Conversely, larger models are deployed for scenarios requiring advanced reasoning capabilities and complex task processing, typically on large-scale servers (Wei et al., 2022).

The conventional approach to constructing a model family involves training each model independently from scratch. However, training large-scale models requires substantial resources, e.g., thousands of GPU days (Touvron et al., 2023). The total computational cost of constructing a model family poses a significant burden on its builders. (Figure 1 (Top)). This fact motivates us to explore more efficient methods for constructing a model family.

We identify **model expansion** as a potential approach to more efficiently constructing model families, including large-scale models. Model expansion is a method of leveraging the parameters of pre-trained smaller models as an effective initialization for larger models (Chen et al., 2022; Du et al., 2024). However, existing work on model expansion typically focuses on

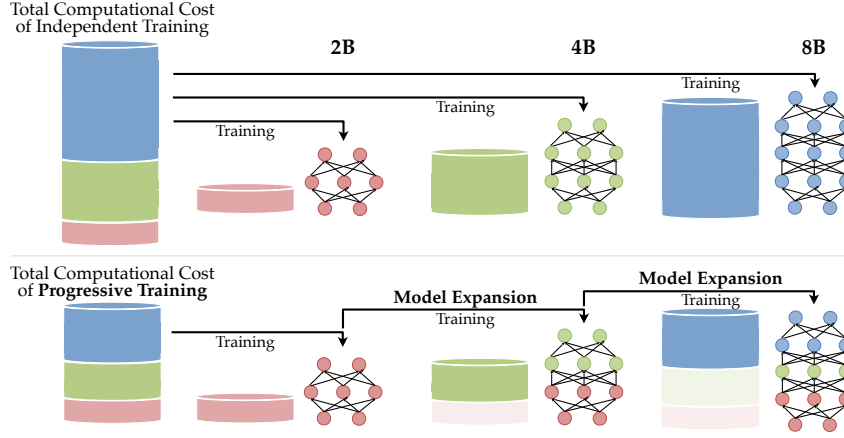


Figure 1: Comparison of approaches for constructing a model family. (*Top*): The conventional approach where each model in the family (2B, 4B, 8B) is trained independently from scratch. The total computational cost equals the sum of individual costs for all models. (*Bottom*): The proposed progressive training utilizes model expansion, where smaller models are expanded to initialize larger ones. The total cost equals only that of the largest model (8B).

producing a single final model, paying less attention to the potential utility of intermediate models. In this work, we empirically explore model expansion as a means to efficiently construct a model family. Specifically, we propose a method that repeatedly applies model expansion to construct models from smaller to larger sizes, namely **progressive training**, thereby reducing the total training cost of constructing a model family (Figure 1 (*Bottom*)).

Through our experiments with a model family ranging from 1B to 8B sizes, we demonstrate that our proposed method reduces the total computational cost of constructing a complete model family by approximately 25% compared to training each model independently¹. Furthermore, by adjusting the maximum learning rate according to model size, our method achieves performance that consistently outperforms independently trained models across various benchmark tasks. Additionally, we show that our progressive training approach produces models with greater behavioral consistency across the model family, as evidenced by lower Kullback-Leibler (KL) divergence between models’ probability distributions.

2 Task Definition and Notation Rules

We consider a model family consisting of models with monotonically increasing sizes and consistent architecture. Formally, we define this model family as a sequence of models of sizes $[X_1, X_2, \dots, X_n]$, corresponding sequence of model parameters $[\theta_1, \theta_2, \dots, \theta_n]$, where $\theta_i \in \mathbb{R}^{X_i}$ represents the parameters of the i -th model. For each model size X_i , let T_i denote the number of tokens used to train the corresponding model. The computational cost (FLOPs) required to train a given model is determined by the model size X_i and the number of tokens T_i . Following Brown et al. (2020), we define FLOPs as: $\text{FLOPs}(X_i, T_i) = 6X_iT_i$. Note that our analysis does not rely on this specific calculation and can be generalized to other reasonable cost approximations (Hoffmann et al., 2022).

Let T_i^{scratch} denote the number of tokens for training a randomly initialized model with size X_i from scratch. Then, the total computational cost of constructing an entire model family is $\sum_{i=1}^n \text{FLOPs}(X_i, T_i^{\text{scratch}})$. The task of this research is to reduce this computational cost and maintain the performance of each model. Formally, we construct each model by satisfying

¹This 25% reduction translates to savings of about 3.2K GPU hours in our experimental setup.

the following constraint:

$$\sum_{i=1}^n \text{FLOPs}(X_i, T_i^{\text{scratch}}) > \sum_{i=1}^n \text{FLOPs}(X_i, T_i^{\text{prog}}), \quad (1)$$

where T_i^{prog} is the number of tokens used for the training model of size X_i in our progressive training approach (Section 3).

3 Method: Progressive Training

We propose a method for constructing language models of multiple sizes more efficiently than the common practice, i.e., training each model independently from scratch. In the common approach, the cost of training a model of size X_i from scratch using T_i^{scratch} tokens² is $\text{FLOPs}(X_i, T_i^{\text{scratch}})$, resulting in the total cost of constructing a model family $\sum_{i=1}^n \text{FLOPs}(X_i, T_i^{\text{scratch}})$. A straightforward approach for satisfying Equation (1) is to reduce the number of training tokens for each model by choosing T_i^{prog} such that $T_i^{\text{scratch}} > T_i^{\text{prog}}$ for $i > 1$. However, a naive reduction of training tokens will lead to performance degradation if other conditions do not change. Instead, when constructing a model family, we jumpstart the training of i -th model of size X_i for $i > 1$ by using a better initialization than random, which is obtained by applying model expansion to its preceding smaller model of size X_{i-1} . We call this procedure **progressive training**.

Specifically, our progressive training starts with training an initial model of size X_1 with T_1^{scratch} tokens. In each subsequent stage, we initialize the next model using a model expansion method f ,

$$\theta_{i+1}^{\text{init}} = f(\theta_i; X_{i+1}) \quad (i \geq 1), \quad (2)$$

where $f(\cdot; X_{i+1}) : \mathbb{R}^{X_i} \rightarrow \mathbb{R}^{X_{i+1}}$ is an off-the-shelf model expansion method. The expanded parameter $\theta_{i+1}^{\text{init}}$ serves as an effective initialization for the subsequent model training.

There is a vast range of possible token allocation patterns $[T_1^{\text{prog}}, T_2^{\text{prog}}, \dots, T_n^{\text{prog}}]$ that satisfy Equation (1)'s condition regarding the total computational cost. In this work, we determine each T_{i+1}^{prog} , ensuring that the total computational cost equals that of training the largest model X_n from scratch:

$$\text{FLOPs}(X_n, T_n^{\text{scratch}}) = \sum_{i=1}^n \text{FLOPs}(X_i, T_i^{\text{prog}}). \quad (3)$$

Specifically, we set T_{i+1}^{prog} so that it satisfies the following FLOPs constraint:

$$\text{FLOPs}(X_{i+1}, T_{i+1}^{\text{prog}}) = \text{FLOPs}(X_{i+1}, T_{i+1}^{\text{scratch}}) - \sum_{j=1}^i \text{FLOPs}(X_j, T_j^{\text{prog}}). \quad (4)$$

Intuitively, we allocate the computational cost of training a model of size X_{i+1} from scratch, subtracting the computational cost already used in previous stages. Through this procedure, we can obtain an entire model family from size X_1 to X_n with the total computational cost, which equals to $\text{FLOPs}(X_n, T_n^{\text{scratch}})$.

How we choose f . In this work, we employ bert2BERT (Chen et al., 2022) as model expansion method for the following reason. Unlike approaches that focus solely on depth expansion (such as stacking (Du et al., 2024)), bert2BERT allows us to increase both the width and depth dimensions of Transformer models (Vaswani et al., 2017) simultaneously,

²Note that the choice of T_i^{scratch} is arbitrary depending on available resources, but most researchers adopt the Chinchilla law (Hoffmann et al., 2022) as a principled approach to determine the compute-optimal number of training tokens. Our experimental setup is also based on the Chinchilla law (Section 4.2).

providing greater flexibility³. Specifically, for width expansion, bert2BERT duplicates linear layer weights to increase hidden dimensions, while for depth expansion, it duplicates pre-trained layers and stacks them⁴. This flexibility to expand in multiple dimensions is particularly valuable as we repeatedly apply model expansion in progressive training.

4 Experiment

To demonstrate the effectiveness of progressive training, we pre-train and compare two model families: one with independently training each model from scratch (Independent) and the other with progressive training (Progressive). Specifically, we show that progressive training improves computational efficiency over independent training and the performance of the two model families is comparable.

4.1 Experimental Setup

We used FineWeb-Edu (Penedo et al., 2024) as the training data for pre-training. We adopted GPT-2 (Radford et al., 2019) tokenizer for tokenization. While progressive training can accommodate any parameter size increase, in this work, we adopted a setting where the parameter count doubles at each stage. Specifically, we construct a model family consisting of $[X_1 = 1\text{B}, X_2 = 2\text{B}, X_3 = 4\text{B}, X_4 = 8\text{B}]$ ⁵. Our models follow the Llama architecture (Touvron et al., 2023). The maximum sequence length for inputs was set to 1024. Furthermore, each model was trained with a cosine learning rate scheduler, using a maximum learning rate of 3.0×10^{-4} .

For the evaluation of each pre-trained model in the family, we used perplexity on the validation data (Valid) from FineWeb-Edu and WikiText (Merity et al., 2017). Additionally, to comprehensively assess the performance of pre-trained models, we measured zero-shot performance across multiple downstream tasks. Specifically, we adopted tasks including language modeling (LAMBADA (Paperno et al., 2016)), commonsense reasoning (Wino-Grande (Sakaguchi et al., 2021), PIQA (Bisk et al., 2020), HellaSwag (Zellers et al., 2019)), and question answering (ARC-e, ARC-c (Clark et al., 2018), OBQA (Mihaylov et al., 2018)).

4.2 Training Data Size and Computational Cost

We prepare the following two data sizes, (i) Chinchilla law (Hoffmann et al., 2022) and (ii) 2x Chinchilla law, to determine the number of training tokens T_i^{scratch} for each model in a model family. Chinchilla law provides the optimal FLOPs required to achieve a specific loss. The guideline is to use 20 tokens per model parameter for training (Hoffmann et al., 2022). 2x Chinchilla law is used to simulate the case such that the amount of training data greatly exceeds the optimal values indicated by the Chinchilla law. In fact, exceeding the Chinchilla law has become a standard practice in recent LLM literature (Sardana et al., 2024).

In the progressive training approach, we determine the number of training tokens T_i^{prog} for each model to satisfy the constraint defined in Equation (4), ensuring that the total computational cost equals that of training the largest model from scratch. For our 2x Chinchilla law setting, this computational requirement resulted in progressive training with the following tokens: $T_1^{\text{prog}} = 40\text{B}$, $T_2^{\text{prog}} = 60\text{B}$, $T_3^{\text{prog}} = 120\text{B}$, $T_4^{\text{prog}} = 240\text{B}$ ⁶.

One of the key advantages of progressive training is the ability to obtain models of multiple sizes efficiently. As defined in Section 2, the computational cost is calculated as FLOPs =

³While recent methods like LEMON (Wang et al., 2024b) have emerged, LEMON builds upon bert2BERT’s foundations with relatively minor modifications to the core mechanisms. We adopt bert2BERT for its simplicity and ease of implementation.

⁴The original paper of bert2BERT (Chen et al., 2022) proposes two variants for model expansion: AKI and FPI. We exclusively use AKI in this work.

⁵The specific width and depth of each model are described in Appendix A.

⁶For the Chinchilla law setting, T_i^{prog} is shown in Appendix B.

	Perplexity ↓		Accuracy ↑						
	Valid	Wikitext	LAMBADA	ARC-e	ARC-c	Winogrande	PIQA	OBQA	HellaSwag
1B Independent	13.14	22.81	39.3	59.3	31.7	54.0	70.5	35.6	46.9
2B Independent	11.30	18.57	45.5	65.2	37.7	55.3	72.1	38.8	54.3
2B Progressive	11.29	18.74	45.1	63.6	36.6	57.4	72.7	39.2	54.7
4B Independent	9.91	15.46	49.5	68.8	41.1	57.2	75.1	43.8	60.5
4B Progressive	9.87	15.12	51.0	69.4	40.0	58.6	75.1	40.6	61.2
8B Independent	8.65	12.24	53.9	71.8	43.1	62.4	76.2	42.6	65.8
8B Progressive	8.61	11.98	55.4	73.5	45.1	62.1	76.6	45.6	67.5

Table 1: Evaluation results of pre-trained models under the Chinchilla law setting. For each model size, we compare the performance of models trained independently from scratch (Independent) versus those built using progressive training (Progressive).

	Perplexity ↓		Accuracy ↑						
	Valid	Wikitext	LAMBADA	ARC-e	ARC-c	Winogrande	PIQA	OBQA	HellaSwag
1B Independent	12.04	20.56	42.0	62.3	34.6	55.5	72.0	35.0	51.6
2B Independent	10.44	17.11	48.5	66.9	38.4	56.6	75.3	41.6	58.0
2B Progressive	10.43	16.94	50.3	70.0	38.1	58.5	73.8	41.4	59.0
4B Independent	9.18	13.99	51.7	71.7	43.2	59.2	76.7	40.8	63.3
4B Progressive	9.00	13.63	51.9	72.6	43.9	61.0	76.1	43.0	65.6
8B Independent	7.85	10.23	55.2	74.5	47.4	62.4	77.0	46.4	67.9
8B Progressive	7.74	10.21	57.8	75.3	46.7	64.6	77.3	44.6	70.2
+Fixed Data	7.73	10.22	56.2	74.8	47.8	63.1	77.8	45.2	70.4

Table 2: Evaluation results of pre-trained models under the 2x Chinchilla law setting. Performance is compared between models trained independently from scratch (Independent) and those built using progressive training (Progressive). Progressive+Fixed Data indicates training on a fixed dataset of 320B tokens.

$6XT$, where X is the parameter count of each model and T is the number of training tokens. For example, independently training each model in the family [1B, 2B, 4B, 8B] under the 2x Chinchilla law would require FLOPs of [0.24Z, 0.96Z, 3.84Z, 15.4Z] respectively. Thus, constructing the entire model family would require 20.4ZFLOPs in total.

In contrast, with progressive training, the computational cost required to construct the entire model family is equivalent to the computational cost required to train the largest-size, i.e., 8B, model. Therefore, the computational cost is 15.4ZFLOPs. Compared to training each model independently, progressive training can reduce computational cost by approximately 25%. This demonstrates that progressive training improves the computational efficiency of constructing a model family.

4.3 Results

Model Performance. Tables 1 and 2 present the evaluation results of pre-trained models in the Chinchilla law and 2x Chinchilla law settings, respectively. Our progressive training approach (Progressive) achieves performance comparable to or better than models that are independently trained from scratch (Independent) across different parameter sizes in both settings. It is particularly noteworthy that, for the largest model (8B), Progressive shows improvements in perplexity and most downstream tasks.

Computational Cost. In Figure 2 (Left), for a given computational cost, i.e., ZFLOPs, Progressive consistently outperforms Independent. In addition, Progressive reaches the training loss of Independent while achieving a 26% reduction in FLOPs.

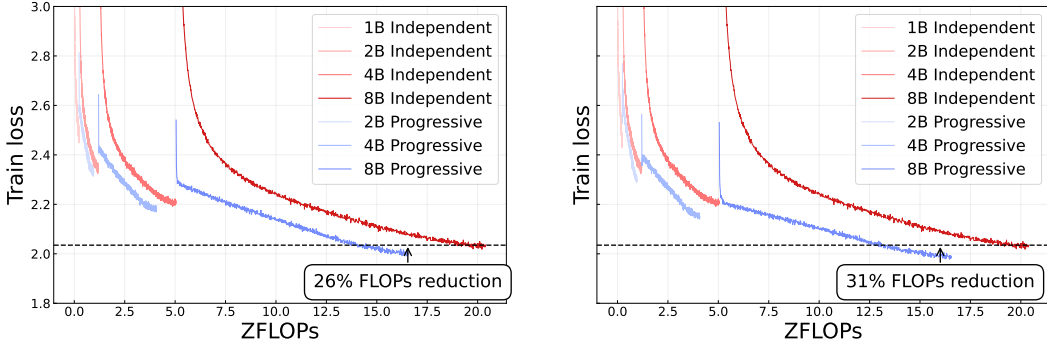


Figure 2: Train loss curves comparing models trained by Independent versus Progressive approach with the 2x Chinchilla law setting. (Left): Models trained with fixed maximum learning rate, achieving 26% FLOPs reduction. (Right): Models trained with maximum learning rate adjustment, from 1.5×10^{-3} (1B) to 3.0×10^{-4} (8B), achieving 31% FLOPs reduction.

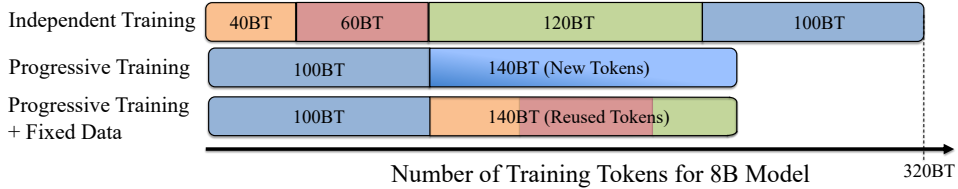


Figure 3: Comparison of token allocation for the 8B model training. (Top): Independent uses a standard approach with 320B tokens. (Middle): Progressive uses 240B tokens (100B + new 140B tokens). (Bottom): Progressive+Fixed Data maintains the same 240B total tokens as Progressive by reusing 140B tokens from previous stages.

Does Progressive Training Exploit More Unique Data? No. For each model size $X_i (i > 1)$ in our model family, we use fewer tokens in progressive training than independent training ($T_i^{\text{prog}} < T_i^{\text{scratch}}$). However, when considering the entire model family construction process, progressive training processes more tokens than when training only the largest model from scratch. As we describe in Section 4.2, the total number of tokens processed during progressive training is $\sum_{i=1}^4 T_i^{\text{prog}} = 460\text{B}$ tokens, which exceeds the $T_4^{\text{scratch}} = 320\text{B}$ tokens used when training the 8B model independently, even though the total computational cost is the same (Figure 3, (Top) and (Middle)). Thus, it is possible that progressive training unfairly benefits from seeing a greater amount of unique training data. To isolate this factor, we introduced an experimental setting called Progressive+Fixed Data. In this setting, we limited the unique training data to exactly 320B tokens, which is the same amount used when independently training the 8B model. Once all 320B tokens were consumed during the training of the 8B model, the remaining $460 - 320 = 140\text{B}$ tokens are reused, i.e., the second epoch begins, as illustrated in Figure 3 (Bottom).

The result is shown in Table 2, in which 8B Progressive+Fixed Data achieves comparable performance. Thus, we confirm that the performance improvements from progressive training do not depend on the addition of more unique data.

4.4 Effectiveness of Maximum Learning Rate Adjustment

In LLM pre-training, learning rate selection involves a careful balance between optimization efficiency and training stability. High learning rates can accelerate convergence and potentially improve final model performance (Takase et al., 2023), but this hyperparameter must be carefully adjusted according to the model size. As models grow larger, they become

		Perplexity ↓		Accuracy ↑						
		Valid Wikitext	LAMBADA	ARC-e	ARC-c	Winogrande	PIQA	OBQA	HellaSwag	
1B	Independent	12.04	20.56	42.0	62.3	34.6	55.5	72.0	35.0	51.6
2B	Independent	10.44	17.11	48.5	66.9	38.4	56.6	75.3	41.6	58.0
	Progressive	10.07	16.17	52.6	71.8	42.2	61.9	75.0	40.8	62.5
4B	Independent	9.18	13.99	51.7	71.7	43.2	59.2	76.7	40.8	63.3
	Progressive	8.72	13.13	55.7	74.5	47.8	65.2	77.4	45.8	68.1
8B	Independent	7.85	10.23	55.2	74.5	47.4	62.4	77.0	46.4	67.9
	Progressive	7.64	10.10	59.0	76.7	48.3	65.8	78.3	46.6	71.0
	+Fixed Data	7.64	9.77	58.0	76.1	50.6	64.7	78.2	47.2	71.2

Table 3: Evaluation results of pre-trained models with maximum learning rate adjustment strategy described in Section 4.4. Progressive models consistently outperform Independent models. +Fixed Data indicates training on a fixed dataset of 320B tokens.

increasingly sensitive to learning rate settings, typically requiring smaller learning rates to maintain training stability (Touvron et al., 2023; Wortsman et al., 2024). For example, in our preliminary experiments, we observed that a learning rate of 1.5×10^{-3} enabled effective and stable training for a 1B model. However, when we applied this same learning rate to an 8B model, we encountered loss value spikes that ultimately led to training collapse⁷.

The progressive training method has the characteristic of starting training from smaller models. Therefore, it is possible to employ high learning rates to enhance performance during small-model training while lowering the learning rate for larger models to stabilize training. In this experiment, we apply this maximum learning rate adjustment to our 2x Chinchilla law setting, linearly decreasing the maximum learning rate from 1.5×10^{-3} for the 1B model to 3.0×10^{-4} for the 8B model, in accordance with the increase in model size, while maintaining a cosine decay schedule during training for all models⁸.

Table 3 presents the results of our maximum learning rate adjustment. Progressive consistently outperforms the Independent counterparts, with more substantial improvements than those observed with no learning rate adjustments (Table 2). As illustrated in Figure 2 (Right), our maximum learning rate adjustment leads to more efficient convergence throughout the training process, achieving 31% FLOPs reduction compared to 26% reduction with no learning rate adjustments.

These results demonstrate the effectiveness of our progressive training approach and how appropriate learning rate adjustments can lead to additional performance gains⁹.

5 The Consistency across Model Family

In this section, we explore the consistency of model behaviors across different sizes in our progressively trained model family. We first discuss the advantages of maintaining consistent behaviors across the model family (Section 5.1). We then present empirical analyses of consistency through probability distribution, quantified using KL divergence between adjacent model sizes (Section 5.2). Our findings demonstrate that models trained through our progressive training approach exhibit greater consistency compared to independently trained models.

⁷A visualization of this training instability with the 8B model is provided in the Appendix C.

⁸The learning rates for each model size are described in Appendix D.

⁹Furthermore, we demonstrate in Appendix E that the effectiveness of our approach persists under post-training setups (SFT+DPO), confirming its robustness for practical applications.

Model Pair	Training Approach	$D_{\text{KL}}(P_{X_i} P_{X_{i+1}})$
1B→2B	Independent	0.2821
	Progressive	0.2162
	Progressive+LR adjustment	0.2538
2B→4B	Independent	0.3087
	Progressive	0.2265
	Progressive+LR adjustment	0.2542
4B→8B	Independent	0.4584
	Progressive	0.3378
	Progressive+LR adjustment	0.3518

Table 4: KL divergence between adjacent model sizes in the different model families. Lower values indicate greater consistency. Results are shown for models trained from scratch (Independent), models built through progressive training (Progressive), and models built with maximum learning rate adjustment (Progressive+LR adjustment).

5.1 The Potential Effectiveness of Consistency across Model Family

Ensuring consistency of behaviors across a model family offers several practical benefits. First, it simplifies deployment flexibility by ensuring that switching between different parameter sizes does not lead to large, sudden shifts in model outputs or user experience. This is particularly valuable when developers need to adapt a model size to varying computational budgets or runtime constraints, as they can easily scale up or down (e.g., from a smaller to a larger model) without retraining users or extensively revalidating system performance (Srivastava et al., 2020; Echterhoff et al., 2024).

Second, this consistency facilitates more efficient incremental improvements or patches across the entire model family. For instance, if developers collect preference data or build a reward model tailored to one member’s outputs (e.g., by annotating the specific texts that this member tends to generate), they inevitably face a distribution mismatch when reusing these artifacts for other models. However, in a consistent family, differences in generation patterns are small enough that the annotated examples remain sufficiently effective for other models (Guo et al., 2024; Zhou et al., 2024a; Tajwar et al., 2024). Likewise, implementations such as safety or content filters calibrated for one model will likely transfer to others with only minimal adjustments (Inan et al., 2023).

Third, consistency between models of different sizes benefits speculative decoding (Leviathan et al., 2023), a technique designed to accelerate inference by letting a smaller model propose “draft” outputs that the larger model then accepts or refines. When the smaller and larger models share similar probability distributions, the larger model is more likely to approve the smaller model’s drafts, reducing the number of rejections and regenerations and thereby improving overall inference speed (Zhou et al., 2024b). Consequently, consistent behavior across a range of model sizes further contributes to more efficient and user-friendly deployments in real-world applications.

5.2 Probability Distribution Consistency across Model Family

The consistency of the underlying probability distributions across the model family provides deeper insights into how knowledge and behaviors propagate. Probability distribution consistency indicates that models share similar processes when generating text.

Experimental Setup. To evaluate the probability distribution consistency across our model family, we measured the KL divergence between adjacent model sizes. KL divergence quantifies how one probability distribution differs from another reference distribution, with lower values indicating greater similarity. For this analysis, we used 10,000 examples from the FineWeb-Edu validation dataset. We calculated the KL divergence $D_{\text{KL}}(P_{X_i} || P_{X_{i+1}})$ between adjacent models by examining their next-token prediction distributions, where P_{X_i}

represents the probability distribution over the vocabulary given by model of size X_i ¹⁰. We conducted this analysis for three different model families: (i) Models trained independently from scratch, (ii) Models built through progressive training, (iii) Models built through progressive training with maximum learning rate adjustment.

Results. Table 4 presents the KL divergence between adjacent model sizes for our three different model families. We found that, across all model pairs, models built through progressive training demonstrate substantially lower KL divergence compared to independently trained models, indicating higher consistency in their probability distributions. This trend is consistent across all adjacent model pairs (1B→2B, 2B→4B, 4B→8B).

This finding provides evidence that our progressive training approach constructs a more coherent model family with consistent internal behavior. The lower KL divergence values suggest that progressively trained models share similar probability distributions and decision-making processes, offering practical advantages for techniques like speculative decoding where alignment between smaller and larger models is crucial.

6 Related Work

Model Family. Offering models in various sizes has become standard practice in language model development to address diverse computational requirements. Major research labs have released model family with multiple parameter sizes: Llama (Touvron et al., 2023; AI@Meta, 2024), Qwen (Yang et al., 2024a), Gemma (GemmaTeam, 2024), and others. Scaling laws (Kaplan et al., 2020; Hoffmann et al., 2022) provide the theoretical foundation for these families, establishing relationships between model size, data, compute, and performance. Recent research highlights the complementary roles of models at different sizes (Wang et al., 2024a); while large models excel in zero/few-shot generalization, smaller models offer advantages in latency-sensitive applications, edge deployment, domain-specific tasks, and privacy-sensitive contexts. This functional differentiation underscores the importance of efficient methods for constructing a coherent model family that maintains consistent capabilities across different parameter sizes. However, the conventional approach of training each model size independently incurs substantial computational costs that scale additively with the number of models in the family. Our work addresses this inefficiency by proposing progressive training that significantly reduces the total computation required to construct a complete, coherent model family.

Model Expansion. Model expansion has emerged as an approach to reduce computational costs when training large-scale models. These methods leverage parameters of smaller, pre-trained models to initialize larger ones, accelerating training. The bert2BERT (Chen et al., 2022) adapted function-preserving methods to Transformers, reducing pre-training costs by approximately 45% for BERT models. LEMON (Wang et al., 2024b) refined this approach with an optimized learning rate scheduler for larger models. For scaling LLMs, Du et al. (2024) identified depthwise stacking as particularly effective, achieving 54.6% speedup for 7B-parameter models. However, these methods typically focus on the efficient training of a single, final model rather than the training of an entire model family. For example, many methods demonstrate the effectiveness of the expansion in a single-shot manner, i.e., applying model expansion only once. In addition, these methods do not take the performance of intermediate model(s) into account. In contrast, we propose progressive training and demonstrate that the off-the-shelf model expansion methods can be used to efficiently train a model family. The progressive training can incorporate emerging novel model expansion techniques, further enhancing efficiency.

7 Conclusion

In this paper, we propose an efficient method for constructing a model family through progressive training, where smaller models are incrementally expanded to larger sizes.

¹⁰The detailed calculation method for KL divergence is described in Appendix F.

Through comprehensive experiments with the model family ranging from 1B to 8B parameters, we demonstrated that our approach significantly reduces the total computational cost required to construct a complete model family by approximately 25% compared to training each model independently. Our method, particularly when combined with maximum learning rate adjustments tailored to each model size, not only matches but exceeds the performance of independently trained models. Furthermore, models built through our progressive approach demonstrate greater consistency across different sizes, as evidenced by lower KL divergence between probability distributions.

This work provides researchers and practitioners with a computationally efficient approach to constructing a high-performing, coherent model family, offering particular value in resource-constrained environments where training multiple models at different scales is desirable.

References

- Marah Abdin, Jyoti Aneja, Hany Awadalla, Ahmed Awadallah, Ammar Ahmad Awan, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Jianmin Bao, Harkirat Behl, et al. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219*, 2024.
- AI@Meta. The llama 3 herd of models, 2024. URL <https://arxiv.org/abs/2407.21783>.
- Yonatan Bisk, Rowan Zellers, Ronan bras, Jianfeng Gao, and Choi Yejin. Piqa: Reasoning about physical commonsense in natural language. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34:7432–7439, 04 2020. doi: 10.1609/aaai.v34i05.6239.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- Cheng Chen, Yichun Yin, Lifeng Shang, Xin Jiang, Yujia Qin, Fengyu Wang, Zhi Wang, Xiao Chen, Zhiyuan Liu, and Qun Liu. bert2BERT: Towards reusable pretrained language models. In Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (eds.), *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 2134–2148, Dublin, Ireland, May 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.acl-long.151. URL <https://aclanthology.org/2022.acl-long.151/>.
- Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge, 2018.
- Ganqu Cui, Lifan Yuan, Ning Ding, Guanming Yao, Bingxiang He, Wei Zhu, Yuan Ni, Guotong Xie, Ruobing Xie, Yankai Lin, Zhiyuan Liu, and Maosong Sun. ULTRAFEEDBACK: Boosting language models with scaled AI feedback. In *Forty-first International Conference on Machine Learning*, 2024. URL <https://openreview.net/forum?id=B0orDpKHjJ>.
- Ning Ding, Yulin Chen, Bokai Xu, Yujia Qin, Shengding Hu, Zhiyuan Liu, Maosong Sun, and Bowen Zhou. Enhancing chat language models by scaling high-quality instructional conversations. In Houda Bouamor, Juan Pino, and Kalika Bali (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 3029–3051, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.183. URL <https://aclanthology.org/2023.emnlp-main.183/>.
- Wenyu Du, Tongxu Luo, Zihan Qiu, Zeyu Huang, Yikang Shen, Reynold Cheng, Yike Guo, and Jie Fu. Stacking your transformers: A closer look at model growth for efficient LLM pre-training. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=FXJDcriMYH>.

- Jessica Maria Echterhoff, Fartash Faghri, Raviteja Vemulapalli, Ting-Yao Hu, Chun-Liang Li, Oncel Tuzel, and Hadi Pouransari. MUSCLE: A model update strategy for compatible LLM evolution. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 7320–7332, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-emnlp.430. URL <https://aclanthology.org/2024.findings-emnlp.430/>.
- GemmaTeam. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295*, 2024.
- GemmaTeam. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*, 2025.
- Shangmin Guo, Biao Zhang, Tianlin Liu, Tianqi Liu, Misha Khalman, Felipe Llinares, Alexandre Rame, Thomas Mesnard, Yao Zhao, Bilal Piot, et al. Direct language model alignment from online ai feedback. *arXiv preprint arXiv:2402.04792*, 2024.
- Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katherine Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Oriol Vinyals, Jack William Rae, and Laurent Sifre. An empirical analysis of compute-optimal large language model training. In Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho (eds.), *Advances in Neural Information Processing Systems*, 2022. URL <https://openreview.net/forum?id=iBBcRU10APR>.
- Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, et al. Llama guard: LLM-based input-output safeguard for Human-AI conversations. *arXiv preprint arXiv:2312.06674*, 2023.
- Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pp. 19274–19286. PMLR, 2023.
- Yu Meng, Mengzhou Xia, and Danqi Chen. SimPO: Simple preference optimization with a reference-free reward. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=3Tzcot1LKb>.
- Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *International Conference on Learning Representations*, 2017. URL <https://openreview.net/forum?id=Byj72udxe>.
- Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. Can a suit of armor conduct electricity? a new dataset for open book question answering. In Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun’ichi Tsujii (eds.), *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pp. 2381–2391, Brussels, Belgium, October-November 2018. Association for Computational Linguistics. doi: 10.18653/v1/D18-1260. URL <https://aclanthology.org/D18-1260>.
- Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Ngoc Quan Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. The LAMBADA dataset: Word prediction requiring a broad discourse context. In Katrin Erk and Noah A. Smith (eds.), *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1525–1534, Berlin, Germany, August 2016. Association for Computational Linguistics. doi: 10.18653/v1/P16-1144. URL <https://aclanthology.org/P16-1144>.

- Guilherme Penedo, Hynek Kydlíček, Loubna Ben allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The FineWeb datasets: Decanting the Web for the finest text data at scale. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2024. URL <https://openreview.net/forum?id=n6SCKn2QaG>.
- Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners, 2019.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=HPuSIXJaa9>.
- Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106, 2021.
- Nikhil Sardana, Jacob Portes, Sasha Doubov, and Jonathan Frankle. Beyond chinchilla-optimal: Accounting for inference in language model scaling laws. In *Forty-first International Conference on Machine Learning*, 2024. URL <https://openreview.net/forum?id=0bmXrtTDUu>.
- Megha Srivastava, Besmira Nushi, Ece Kamar, Shital Shah, and Eric Horvitz. An empirical analysis of backward compatibility in machine learning systems. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, KDD '20, pp. 3272–3280, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450379984. doi: 10.1145/3394486.3403379. URL <https://doi.org/10.1145/3394486.3403379>.
- Fahim Tajwar, Anikait Singh, Archit Sharma, Rafael Rafailov, Jeff Schneider, Tengyang Xie, Stefano Ermon, Chelsea Finn, and Aviral Kumar. Preference fine-tuning of LLMs should leverage suboptimal, on-policy data. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- Sho Takase, Shun Kiyono, Sosuke Kobayashi, and Jun Suzuki. Spike no more: Stabilizing the pre-training of large language models. *arXiv preprint arXiv:2312.16903*, 2023.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- Fali Wang, Zhiwei Zhang, Xianren Zhang, Zongyu Wu, Tzuhao Mo, Qiuhao Lu, Wanjing Wang, Rui Li, Junjie Xu, Xianfeng Tang, et al. A comprehensive survey of small language models in the era of large language models: Techniques, enhancements, applications, collaboration with LLMs, and trustworthiness. *arXiv preprint arXiv:2411.03350*, 2024a.
- Yite Wang, Jiahao Su, Hanlin Lu, Cong Xie, Tianyi Liu, Jianbo Yuan, Haibin Lin, Ruoyu Sun, and Hongxia Yang. LEMON: Lossless model expansion. In *The Twelfth International Conference on Learning Representations*, 2024b. URL <https://openreview.net/forum?id=3Vw7DQqq7U>.
- Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. Emergent abilities of large language models. *Transactions on Machine Learning Research*, 2022. ISSN 2835-8856. URL <https://openreview.net/forum?id=yzkSU5zdwD>. Survey Certification.

Mitchell Wortsman, Peter J Liu, Lechao Xiao, Katie E Everett, Alexander A Alemi, Ben Adlam, John D Co-Reyes, Izzeddin Gur, Abhishek Kumar, Roman Novak, Jeffrey Pennington, Jascha Sohl-Dickstein, Kelvin Xu, Jaehoon Lee, Justin Gilmer, and Simon Kornblith. Small-scale proxies for large-scale transformer training instabilities. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=d8w0pmvXbZ>.

An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jianxin Yang, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, Mingfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Xuejing Liu, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, Zhifang Guo, and Zhihao Fan. Qwen2 technical report, 2024a. URL <https://arxiv.org/abs/2407.10671>.

An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen2. 5 technical report. *arXiv preprint arXiv:2412.15115*, 2024b.

Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In Anna Korhonen, David Traum, and Lluís Màrquez (eds.), *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 4791–4800, Florence, Italy, July 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1472. URL <https://aclanthology.org/P19-1472>.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2023. URL <https://openreview.net/forum?id=ucCHPGDlao>.

Wenxuan Zhou, Ravi Agrawal, Shujian Zhang, Sathish Reddy Indurthi, Sanqiang Zhao, Kaiqiang Song, Silei Xu, and Chenguang Zhu. WPO: Enhancing RLHF with weighted preference optimization. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, November 2024a. doi: 10.18653/v1/2024.emnlp-main.475. URL <https://aclanthology.org/2024.emnlp-main.475/>.

Yongchao Zhou, Kaifeng Lyu, Ankit Singh Rawat, Aditya Krishna Menon, Afshin Rostamizadeh, Sanjiv Kumar, Jean-François Kagy, and Rishabh Agarwal. Distillspec: Improving speculative decoding via knowledge distillation. In *The Twelfth International Conference on Learning Representations*, 2024b. URL <https://openreview.net/forum?id=rsY6J3ZaTF>.

A Details of Experimental Configurations

In our experiments (Section 4), we constructed models with 1B, 2B, 4B, and 8B parameters based on the Llama architecture. Table 5 shows the specific experimental configurations for each model size. As the model expands, we gradually increase the hidden dimension, FFN dimension, number of layers, and number of heads.

B Data Size for (2x) Chinchilla Law Setting

As described in Section 4.2, we determined the number of training tokens for each model size according to Hoffmann et al. (2022). We report token allocations for each model in Table 6 (Chinchilla law setting) and Table 7 (2x Chinchilla law setting).

Configuration	1B	2B	4B	8B
Hidden dimension	2048	2560	3200	4096
FFN dimension	7168	8960	11200	14336
Layers	18	22	27	33
Heads	16	20	25	32
Batch size	960	1920	3840	7680
The number of updates	40700	40700	40700	40700
learning rate warmup fraction	0.01	0.01	0.01	0.01
Learning rate decay style	cosine	cosine	cosine	cosine
Adam β_1	0.9	0.9	0.9	0.9
Adam β_2	0.95	0.95	0.95	0.95
Gradient clipping	1.0	1.0	1.0	1.0
Weight decay	0.1	0.1	0.1	0.1
Precision	bfloat16	bfloat16	bfloat16	bfloat16

Table 5: Experimental configuration for each model size

Model Size	T_i^{scratch} (Independent)	T_i^{prog} (Progressive)
1B	20B	20B
2B	40B	30B
4B	80B	60B
8B	160B	120B

Table 6: Training token allocation for Independent and Progressive the Chinchilla law setting.

C 8B Model Loss Visualization

Figure 4 illustrates the training loss curves for 8B models trained from scratch with different learning rates: 3.0×10^{-4} and 1.5×10^{-3} . While the lower learning rate of 3.0×10^{-4} results in stable training, the higher learning rate of 1.5×10^{-3} causes significant loss spikes during training.

D Specific Configurations in the Learning Rate Adjustment Strategy

In our learning rate adjustment strategy (Section 4.4), we decrease the maximum learning rate as the model size increases. We apply maximum learning rates of 1.5×10^{-3} , 1.1×10^{-3} , 7.0×10^{-4} , and 3.0×10^{-4} for the 1B, 2B, 4B, and 8B models, respectively. This gradual reduction accommodates the increasing sensitivity of larger models to high learning rates.

E Experiment: Model Family Post-Training

This experiment verifies whether the trends observed in pre-training experiments in Section 4 are also seen in post-training. We examine whether models built using progressive training with fixed learning rates can achieve comparable performance to models trained from scratch. In addition, we confirm that performance improvements observed with maximum learning rate adjustments in pre-training also extend to post-training.

E.1 Experimental Setup

We evaluate the effectiveness of progressive training by conducting post-training on the model family constructed in Section 4. We adopted the settings from Meng et al. (2024) for post-training. Specifically, we performed Supervised Fine-Tuning (SFT) using the UltraChat-200k dataset (Ding et al., 2023), followed by DPO (Rafailov et al., 2023) using the Ultrachat Feedback dataset (Cui et al., 2024).

Model Size	T_i^{scratch} (Independent)	T_i^{prog} (Progressive)
1B	40B	40B
2B	80B	60B
4B	160B	120B
8B	320B	240B

Table 7: Training token allocation for Independent and Progressive the 2x Chinchilla law setting.

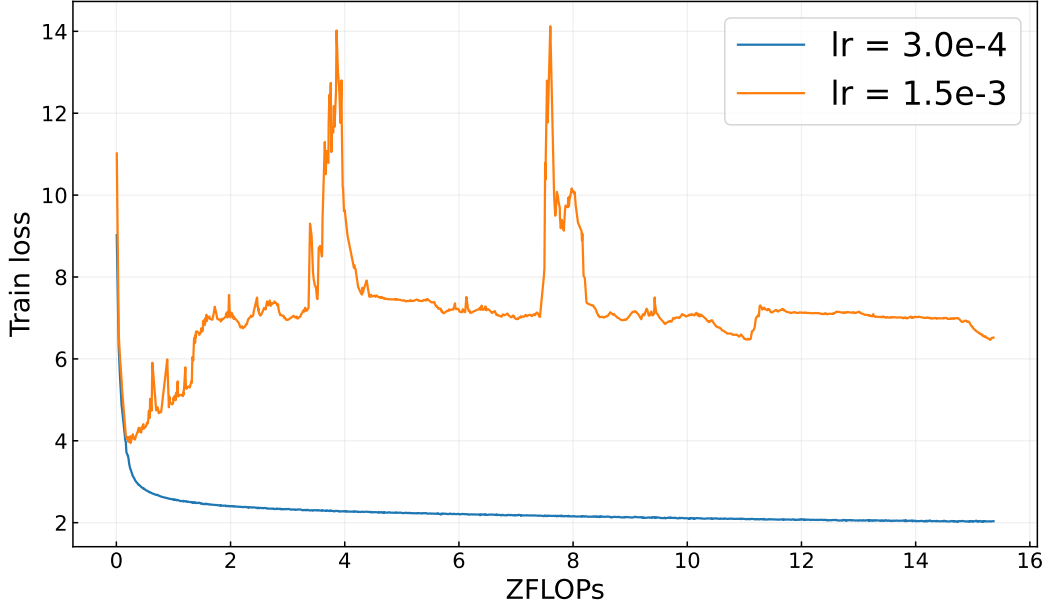


Figure 4: Training loss curves comparing 8B models trained from scratch with different learning rates. The model trained with a learning rate of 3.0×10^{-4} (blue line) shows stable training, while the model trained with a higher learning rate of 1.5×10^{-3} (orange line) exhibits severe loss spikes indicating training instability.

The maximum learning rate was set to 3.0×10^{-5} for SFT and 5.0×10^{-7} for DPO, standardized across all models. We evaluated the response quality of each model using MT-Bench (Zheng et al., 2023) using gpt-4-0613 as an evaluator. For comparison, we prepared models (2B, 4B, 8B) independently trained from scratch and conducted the same post-training.

E.2 Results

Table 8 presents the MT-Bench evaluation results after post-training across different training approaches. With fixed learning rates, models trained with progressive training perform comparably to models trained from scratch, with some variation across model sizes.

When applying our maximum learning rate adjustment, progressive training consistently outperforms from-scratch training across all model sizes, with substantial improvements observed. The models with maximum learning rate adjustment significantly outperform those with fixed maximum learning rates for progressive training.

These results indicate that our progressive training approach, particularly when combined with maximum learning rate adjustment, can deliver both computational efficiency and enhanced model performance after post-training.

Size	Training Approach	Chinchilla MT-Bench $\uparrow$	2x Chinchilla MT-Bench $\uparrow$	Adjusted LR 2x Chinchilla MT-Bench $\uparrow$
2B	Independent	1.61	2.04	2.04
	Progressive	2.03	2.46	3.22
4B	Independent	2.78	3.02	3.02
	Progressive	2.87	3.01	3.63
8B	Independent	3.42	3.45	3.45
	Progressive	3.28	3.21	3.96
	Progressive+Fixed Data	3.32	3.41	3.98

Table 8: MT-Bench evaluation results after post-training (SFT+DPO) for models trained with different approaches. We compare models trained from scratch (Independent) versus those built using our progressive training approach (Progressive) under both fixed maximum learning rate and learning rate adjustment (Adjusted LR) strategies. For the fixed learning rate, results are shown for both the Chinchilla law and 2x Chinchilla law settings. For the models with learning rate adjustments, the maximum learning rates are decreased from 1.5×10^{-3} for 1B to 3.0×10^{-4} for 8B as the model size increases.

F KL Divergence Computation

To simplify the computation, at each token position, we computed $D_{\text{KL}}(P_{X_i} \parallel P_{X_{i+1}}) = \sum_{v \in V} P_{X_i}(v) \cdot (\log P_{X_i}(v) - \log P_{X_{i+1}}(v))$, where V is the vocabulary and $P_{X_i}(v)$ is the probability assigned to vocabulary token v by model X_i . We first averaged these KL divergence values across all token positions within each example, then computed the final KL divergence by averaging across all 10,000 examples from the FineWeb-Edu validation dataset, as described in Section 5.2.